NIOS Last-Minute Study Assistant – Full Project Document

1. Product vision

- **What this is**:

A NIOS-focused last-minute study assistant for Classes 10 & 12 that builds

PYQ-based, goal-driven learning paths

from

official NIOS PDFs

plus past year question papers, powered by

Cloudflare AI

and

serverless functions

.

- **Core promise to students**:

"Tell us your subjects, exam date, and goal. We'll give you **daily 20–60 minute plans** built from **real NIOS material + PYQs**, with **short notes and explanations in English/Hindi/Hinglish**, optimized for *just pass* or *score higher*."

---

2. Target users & problems

- **Users**
- NIOS Class 10 / 12 students.
- Often late starters, working students, or those who failed before.

- **Key problems**
- Don’t know **what to focus on** in limited time.
- NIOS PDFs are **long and dense**, difficult to skim.
- Hard to connect **theory from PDFs** to **actual PYQs**.
- Need **language-flexible explanations** (English, Hindi, Hinglish).
- **This product solves**
- Converts PDFs + PYQs into **small, guided tasks**.
- Links every task to **exam pattern and past questions**.
- Uses AI to **simplify, translate, and personalize** explanations.

---

3. High-level solution overview

- **Input**
- Official NIOS syllabus PDFs and textbooks (free).
- NIOS PYQs from previous years (structured by subject, year, marks).
- **Core idea**
- Break content into Subject → Chapter → Topic → Concept.
- Use **Cloudflare AI** to:
- Summarize topics.
- Generate explanations for PYQs.
- Create “common mistakes” and “why this matters” sections.
- Re-explain concepts in Hinglish/Hindi on demand.
- Use **simple rules** + basic analytics to generate **learning paths**:
- Based on exam date, daily time, and goal (Just Pass / 60% / 80%+).
- **Delivery**
- **Web app** (mobile-first UI) + **mobile app** (React Native/Flutter).
- Both consume the same **REST APIs** from serverless backend.

---

4. Core feature set

4.1 Onboarding & personalization

- Student inputs:
- Class (10/12)
- Subjects (e.g., Maths, English, Science, etc.)
- Exam month/year
- Daily available time (e.g., 20, 40, 60 min)
- Goal: **Just Pass**, **60%+**, **80%+**
- Preferred explanation language: **English / Hindi / Hinglish**
- System outputs:
- A multi-week **learning path** per subject.
- A **daily plan** of tasks (example: “Read Topic X notes (10 min)”, “Solve 8 PYQs on Topic Y (15 min)”).

4.2 Subject & topic explorer

- Per subject:
- Syllabus breakdown by **chapter** → **topic**.
- For each topic:
- Short notes (bullet summaries).
- Linked PYQs with answers & explanations.
- Difficulty & frequency tags (e.g., “High frequency in last 5 years”).
- Estimated time to cover.

4.3 PYQ-first practice

- Topic-wise PYQ sets:
- **Question card** (question text, marks, year).
- **Reveal answer & explanation** (AI-enhanced but syllabus-bound).
- Mark as **Correct / Incorrect**, optional bookmark.

- Small **chapter tests** combining PYQs from multiple topics.

4.4 Daily plan & progress tracking

- “Today’s Plan” screen:
- Tasks grouped by subject.
- Each task: label, topic, estimated time, completion checkbox.
- Progress views:
- % topics covered per subject.
- Accuracy in PYQs.
- Simple indicator: “On track / Slightly behind / At risk”.

4.5 AI-powered assist

- **Explain again**: Rephrase explanation in:
- Simpler language.
- Hinglish.
- Hindi.
- **Concept doubt**: Small text box:
- Constrained Q&A using **only** related NIOS content and your notes as context.
- **Micro-motivation**:
- AI-generated 1–2 line encouragements (optional, cached templates).

---

5. Content pipeline (from NIOS PDFs & PYQs)

5.1 Ingestion of NIOS PDFs

- For each subject:
- Download official NIOS PDFs.
- Preprocess:
- Extract text per **chapter** and **section headings**.
- Create **topic chunks** of manageable size (e.g., 300–800 words).

- Store:
- Raw PDF file in object storage (e.g., Cloudflare R2).
- Extracted structured content in DB:
- Subject, Chapter, Topic, SourceSection.

5.2 PYQ collection

- Collect past year papers:
- For each question:
- Subject, year, exam session, marks, section.
- Manual/assisted mapping to Topic and QuestionType (MCQ / short / long).
- Store:
- In a PYQ table with tags: difficulty, recurrence frequency, etc.

5.3 AI-generated pedagogical layers

Using **Cloudflare AI** in batch/offline mode:

For each topic:

- **Topic summary** (English):

"Explain this topic in 10–15 bullet points for a NIOS student. Use simple language."

- **Why this topic is important**:

Short 2–3 line explanation linking to exam.

- **Common mistakes**:
- 3–5 common conceptual or calculation errors.
- **Language variants**:
- Translate summaries & explanations to:
- Hindi.
- Hinglish (informal but clear).

For each PYQ:

- **Step-by-step explanation**:

- Based only on the topic content + official textbook concepts.
- **Hints**:
- 1–2 hints before showing full solution.

All these are stored in DB (or KV) so that **runtime AI calls are minimal**.

---

6. Learning path & goal logic

6.1 Inputs

- Exam date → days left.
- Daily study time (minutes).
- Subjects selected.
- Target (Pass / 60% / 80%+).
- Current progress & accuracy stats.

6.2 Rules (V1 – rule-based)

- **Just pass**
- Cover ~60–70% of the total expected marks.
- Prioritize:
- Topics with **high PYQ frequency**.
- Medium difficulty.
- Skip very low-yield / fringe topics if time is short.
- **60%+**
- Cover all high & medium-yield topics.
- Add more practice sets + 1–2 revision rounds.
- **80%+**
- Cover almost all topics, with extra:
- Hard PYQs.
- Mixed-chapter tests.

- Error review sessions.

6.3 Task generation

- Break each day into **10–20 minute tasks**:
- READ_NOTES: Topic notes (estimated 10–15 min).
- PRACTICE_PYQ_SET: 5–10 questions (15–20 min).
- REVISE_WRONGS: Revisit previously wrong questions.
- Algorithm (simplified):
- Estimate total available minutes (days * daily minutes).
- Allocate minutes per subject based on:
- Number of chapters.
- User's weak areas (from accuracy).
- Build sequence of tasks per subject and spread them across days.
- Regenerate/reschedule if user falls behind (simple shifting of uncompleted tasks).

---

7. System architecture

7.1 High-level components

Mermaid Syntax Error

View diagram source

7.2 Deployment choices

- **Hosting**
- Cloudflare Pages for static frontends (web).
- Cloudflare Workers for APIs.
- **DB**
- Cloudflare D1 or any free-tier Postgres.
- **File storage**
- Cloudflare R2 (for PDFs and potential preprocessed JSONs).

- **AI**
- Cloudflare AI (Workers AI) for:
- Content summarization.
- Explanation generation.
- On-demand doubt resolution.

---

8. Tech stack

- **Frontend (web)**
- React / Next.js (mobile-first design).
- Authentication via OTP or simple token-based.
- **Mobile**
- React Native or Flutter.
- Shared API client with web.
- **Backend**
- Cloudflare Workers (TypeScript/JavaScript).
- REST APIs (JSON).
- **Data**
- D1 / Postgres for relational data.
- KV / R2 for caching AI outputs & files.
- **AI integration**
- Cloudflare Workers AI with carefully designed prompts:
- Always pass explicit **context** (topic text, notes, official PDF excerpts).
- Strong instruction: **do not invent new syllabus content**.

---

9. Data model (simplified)

9.1 Main tables

- **User**

- id
- class_level (10/12)
- goal (“pass”, “60”, “80”)
- exam_date
- daily_minutes
- preferred_language (en/hi/hinglish)
- **Subject**
- id, name, class_level
- **Chapter**
- id, subject_id, title, order_index
- **Topic**
- id, chapter_id, title, order_index
- high_yield_score
- **TopicContent**
- topic_id
- lang (en/hi/hinglish)
- summary_bullets (JSON/string)
- why_important
- common_mistakes
- **PYQ**
- id
- subject_id, topic_id
- year, session
- question_text
- marks
- difficulty

- frequency_score
- **PYQExplanation**
- pyq_id
- lang (en/hi/hinglish)
- steps
- hints
- **LearningPath**
- id, user_id, subject_id, generated_at
- **LearningPathStep**
- id, learning_path_id
- day_index
- task_type (READ_NOTES / PRACTICE_PYQ_SET / REVISE_WRONGS)
- topic_id
- estimated_minutes
- **UserProgress**
- user_id, topic_id
- status (not_started / in_progress / done)
- correct_count, incorrect_count
- last_reviewed_at
- **TaskCompletion**
- user_id, step_id
- completed_at

---

10. API design (examples)

10.1 Onboarding & profile

- POST /api/user/profile

- Body: { classLevel, subjects, examDate, dailyMinutes, goal, language }
- Response: { userId, recommendedPlanSummary }

10.2 Learning path

- POST /api/plan/generate
- Body: { userId }
- Response: DailyPlan[] with tasks for upcoming days.
- GET /api/plan/today?userId=...
- Response: { date, tasks: [...] }

10.3 Content

- GET /api/subjects?classLevel=12
- GET /api/subject/:id/chapters
- GET /api/topic/:id/content?lang=hinglish
- GET /api/topic/:id/pyqs

10.4 Practice & progress

- POST /api/practice/attempt
- Body: { userId, pyqId, isCorrect }
- POST /api/task/complete
- Body: { userId, stepId }
- GET /api/progress/overview?userId=...

10.5 AI endpoints (Cloudflare AI behind the scenes)

- POST /api/ai/explain
- Body: { topicId, userId, style: "simple" | "hinglish" | "hindi" }
- Backend:
- Fetch topic content + existing explanations.
- Check cache.
- If not found, call Cloudflare AI with NIOS-based context.

- POST /api/ai/doubt
- Body: { userId, topicId, question }
- Backend:
- Fetch relevant topic text, notes, PYQs.
- Call Cloudflare AI with strict instruction to answer from given context.

---

11. Frontend UX design (web + mobile)

11.1 Shared UX principles

- **Mobile-first layouts** (big tappable cards, minimal text per screen).
- Show **one clear "next action"** per screen.
- Reduce typing: mostly buttons and choices.

11.2 Main screens

- **Onboarding wizard**
- Step-by-step: Class → Subjects → Goal → Exam date → Time per day → Language.
- **Home / Today**
- "Today's Plan" with:
- Tasks sorted by subject.
- Progress bar and total time.
- **Subject detail**
- Chapters list with completion indicators.
- **Topic view**
- Tabbed or scroll sections:
- Notes.
- PYQs.
- Mistakes.
- "Explain again" button.

- **Practice view**
- Single question per screen.
- Navigation: Next/Previous, Mark correct/incorrect.
- **Profile / Settings**
- Change language, goal, daily time.

---

12. Cloudflare AI usage & constraints

- **Batch (offline) usage**
- Run Jobs (through scripts) to:
- Summarize each topic (EN).
- Generate Hindi + Hinglish variants.
- Create common mistakes & “why important”.
- Generate standard explanations for each PYQ.
- **Online (per-user) usage**
- Limit to:
- Re-explanations in requested style.
- Doubt Q&A within topic.
- Cache (KV) by:
- (topicId, language, style) for explanations.
- (topicId, normalized_question_text) for doubts.
- **Prompt design (conceptual)**
- System prompt: “You are a tutor for NIOS students. Use only the provided official NIOS-based content. Do not introduce topics outside this context. Keep answers short and exam-focused.”
- User prompt includes: relevant text from chapter/notes + the question.
- **Cost control**

- Per-user quotas (requests/day).
- Prefer pre-computation + caching to reduce live AI calls.

---

13. Security & privacy basics

- **Data**
- Minimal PII: maybe only phone/email + name.
- Store hashed identifiers where possible.
- **Transport**
- All client–server calls over HTTPS.
- **Auth**
- Simple token-based auth, short-lived, refreshable.
- **AI**
- Do not send raw phone/email/etc. to AI; only study content + anonymized user stats.

---

14. Phased roadmap

- **Phase 0 – Prototype**
- One subject (e.g., Class 12 Maths), 1–2 chapters.
- Manual tagging + basic AI summaries.
- Simple web-only UI.
- **Phase 1 – MVP**
- 2 subjects, 5–6 chapters each.
- Full onboarding + daily plan.
- AI summaries + explanations precomputed.
- Web + basic mobile app sharing APIs.
- **Phase 2 – Growth**
- More subjects & complete coverage.

- Better analytics, improved recommendation logic.
- Offline caching on mobile, better doubt support.